

Document Number: P1137R0
Date: 2018-06-24
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [May 16th, 2018](#)
- [May 30th, 2018](#)
- [June 20th, 2018](#)

May 16th, 2018

Draft agenda:

- Review and discuss papers in the Rapperswil pre-meeting mailing.
- Discuss plans and goals for those attending Rapperswil.

Attendees:

- Bob Steagal
- Corentin Jabot
- Dalton Woodward
- Florin Trofin
- JeanHeyd Meneide
- Mark Zeren
- Martinho Fernandes
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- It was reported on Slack that Martinho's properly formatted UTF-8 [P1041R0](#) paper was served up by open-std.org either without a CharSet header or with a Latin1 setting. Tom contacted Hal and Keld. Further discussion yielded a plan to update [SD-7](#) to require UTF-8 for .md files and to configure the open-std.org web server to serve them with a CharSet=UTF-8 header.
- Zach, Bob, and JeanHeyd shared some of their experience at C++Now.
- We then went on to review papers from the pre-Rapperswil mailing.
- [P1041R0](#) - Make char16_t/char32_t string literals be UTF-16/32
 - Tom noted a typo in the proposed wording changes for `lex.ccon/4`; a use of UTF-8 where UTF-16 was intended.

- Given the encoding issues and lack of Markdown rendering support built into browsers, it was suggested that future papers, at least for now, be submitted in a pre-rendered format.
- Martinho asked about getting the paper scheduled for discussion in Rapperswil. Tom said he would forward SG16 polls on papers we discussed to WG chairs to communicate our position and request time in Rapperswil. Tom will copy paper authors and expected presenters on this communication.
- It was asked if there was any library impact. Martinho responded no. Tom noted having previously audited occurrences of char16_t, char32_t, UTF-16, and UTF-32 and could not think of a case.
- Zach suggested that, when presenting, it be emphasized that no implementation will need to make changes; that this is just standardizing existing practice. Emphasize that there are no known implementations where the encoding used is not already UTF-16/UTF-32 and that a member of the C committee was consulted.
- Poll: Those in favor of P1041R0?
 - Unanimous consent.
- [P1072R0](#) - Default Initialization for basic_string
 - Mark noted that [P1072](#) is dependent on [P1010](#) which is dependent on Richard Smith's [P0593](#). This raised the question of prioritization and a request for SG16 to request that [P0593](#) and [P1010](#) get time in Rapperswil so that progress can be made on [P1072](#) in San Diego. Tom agreed to make such a request; specifically to request that EWG entertain [P0593](#) and that LEWG look at [P1010](#) (and [P1072](#) time permitting).
 - Mark went on to discuss applicability of [P1072](#) to SG16. Of particular concern are the issues caused by requiring null termination. This is not a problem for vector, and hence not a concern expressed in [P1010](#).
 - Mark pointed out that the design is used in real world code today.
 - Zach asked why reserve() doesn't suffice. Mark explained the examples in the paper; that we currently either have to repeatedly update the size of the container with each addition, or eagerly resize the container and pay for an unused initialization. The goal of the paper is to avoid both costs by enabling writing to excess capacity independently of updates to the container size.
 - Tom asked if option A is viable. The concern is that const member functions must be thread safe. A call to resize_uninitialized() makes uninitialized data available to const member functions. Further, there is no event to indicate when the uninitialized data has been read and therefore no memory barrier to function as a synchronization point.
 - Mark acknowledged that a two-phase commit approach is necessary to avoid UB.
 - Martinho observed that two-phase commit is not sufficient by itself because basic_string uses excess capacity to store a null terminator for the string; this is what allows the data() and c_str() member functions to be const qualified. Overwriting the null terminator will cause UB for concurrently executing threads.
 - Mark advised SG16 to consider the consequences of providing implicit null-termination for string-like containers in the future. An alternative approach would use string builders that append a null-terminator when they are collapsed.
 - Mark noted that the two-phase commit approach does at least allow the implementation to re-establish invariants (such as ensuring a null-terminator is present at the start of excess capacity following insert_from_capacity()).
 - Tom suggested an emplace-like solution might be preferred to enable preserving invariants.
 - Mark acknowledged a call-back/functor based solution would work (though it still doesn't address the over-written null-terminator issue).
 - Dalton asked whether making vector/string node-based containers such that data could be written to a new buffer and then swapped in. This has the disadvantage of requiring that the current buffer be copied prior to performing the append.
 - Tom asked if any performance numbers were available. What is the expected gain?
 - Mark responded that numbers are not available, but that Google has measured and claims the improvements make this feature worthwhile. Estimate is a few percent improvement.
 - Corentin asked why not to use vector instead of string.
 - Mark responded that string is a vocabulary type.
 - Poll: Do we agree P1072R0 addresses a problem worth solving?

- Unanimous consent.
- Poll: Do we prefer option A, option B, or some option C?
 - A: 0
 - B: 2
 - C: 5
- Mark clarified that option C, as discussed today, would be one of:
 - An emplace-like call with a call-back/functor.
 - A node-based swap.
- Discussion moved into allocator interaction with node types.
- Zach stated that swap is broken for PMR allocators.
- Steve agreed and provided an elaboration; that the swap of the allocators doesn't swap the actual buffers.
- Mark noted that moving a buffer between vector and string encounters complexities due to null-termination requirements.
- Martinho asked how a small buffer optimized string is moved into a node type.
- Dalton responded that you allocate.
- Tom added, or the node type implements the SBO itself.
- Mark expressed concern that an emplace-like call-back/functor approach may not work for the network use case of wanting to read data off the network directly into the buffer.
- Zach suggested that, in a string builder approach, vector is the string builder.
- Corentin expressed a preference for a specific string builder type rather than vector. Essentially a vocabulary type suited to the purpose.
- [P1025R0](#) - Update The Reference To The Unicode Standard
 - Steve briefly introduced the change as similar to what had been proposed, but not completed, for C++17.
 - Tom asked, why update the normative reference to specify each of Unicode 10, Unicode without a version indicator, and ISO 10646?
 - Steve answered, we need ISO 10646 for existing references; for example, the `__STDC_ISO_10646__` macro. We want to reference the Unicode standard (in addition to ISO 10646) for stability guarantees and additional features. We want to reference Unicode 10 to establish a minimum requirement, and the unversioned Unicode standard to enable implementors to adopt a newer version.
 - Tom suggested adding a non-normative note that implementors are allowed to use Unicode 10 or newer; though they must use a corresponding version of ISO 10646.
 - Martinho stated that we need to make it clear that implementors must choose a specific Unicode release.
 - Tom asked if we should require a predefined macro that indicates the Unicode version.
 - Steve and Martinho both answered, maybe, but not yet as we don't actually depend on anything Unicode version dependent yet.
 - Poll: Those in favor of P1025R0:
 - Unanimous consent.
- Our next meeting will be May 30th; the week before Rapperswil.
- There is a WG21 administrative teleconference May 25th.
 - Tom will dial-in to give an update on SG16. Martinho and JeanHeyd are encouraged to attend as well since they have papers to present.
- Those planning to attend Rapperswil: Martinho, Corentin, Peter, JeanHeyd.
- Following the meeting, Martinho volunteered to present [P1025R0](#) at Rapperswil since Steve will not be present. Steve agreed.

May 30th, 2018

Draft agenda:

- Discuss plans and goals for those attending Rapperswil.
- Review and discuss the following papers from the Rapperswil pre-meeting mailing:
 - P1030R0: `std::filesystem::path_view`
 - P0540R1: A Proposal to Add `split/join` of `string/string_view` to the Standard Library
 - P0645R2: Text Formatting

Attendees:

- JeanHeyd Meneide
- Mark Zeren
- Martinho Fernandes
- Peter Bindels
- Sergey Zubkov
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Administrative updates:
 - Tom reported that WG chairs were contacted regarding SG16 requests for paper reviews in Rapperswil. WG chairs are predictably swamped and prioritizing as best they know how, but we may not get to present any of our papers.
 - Zach observed that Titus is concerned about the amount of time that LEWG will need for ranges, but that LWG should be more concerned.
 - Tom relayed that JF Bastien volunteered to arrange introductions with Swift and WebKit developers working on Unicode. Tom reached out to arrange meetings, but hasn't heard back. Apple developers are busy preparing for WWDC; Tom will reach out again soon.
 - Tom brought up the recent news that Microsoft has added beta support for UTF-8 as a system code page as of the Windows 10 April update. Tom made some new contacts within Microsoft, but has not yet gotten any further information about Microsoft's goals or plans with this change.
- Rapperswil planning:
 - Tom asked for volunteers to standup for SG16 at the Saturday plenary in Rapperswil and give a brief update. Martinho and JeanHeyd agreed to do so.
 - Tom asked for those who have attended meetings before to offer any advice they have for first time attendees.
 - Zach recommended spending some time in each of the WGs. Each WG has its own personality.
 - It was noted that hanging around in WGs where one has a short paper in the queue creates opportunities to present earlier than the paper might otherwise be scheduled. The [P1025](#) (normative Unicode reference) and [P1041](#) (`char16_t/char32_t` are UTF-16/UTF-32) papers are good candidates.
 - Zach also mentioned not to be afraid to ask questions and to try to read papers ahead of time.
 - Tom noted that anyone present in the room is allowed to vote in straw polls, but that polls in plenary are generally restricted to ISO members. It was noted that Herb will make it clear when ISO membership is required to vote.
- [P1030R0](#) - `std::filesystem::path_view`
 - Martinho liked it, especially section 4.1 (Assume UTF-8 for char based interfaces).
 - Tom liked it with the exception of section 4.1.
 - Tom expressed a belief that the discussion in section 4.1 of how existing char based interfaces on Windows handle conversion to `wchar_t` for invocation of native filesystem interfaces is incorrect. Tom's understanding is that char based strings are transcoded to `wchar_t` strings using the system code page.
 - Zach asked what is meant by ANSI encoding.

- Tom explained that Microsoft has long referred to char based encodings collectively as ANSI encodings despite these encodings not reflecting an ANSI standard.
- *[Editor's note: Microsoft's glossary of terms on MSDN describes the origin of the ANSI reference here. It comes from a draft ANSI specification that was eventually standardized as the ISO-8859 family of encodings. See the definition of "ANSI" at <https://msdn.microsoft.com/en-us/goglobal/bb964658.aspx#a>. Microsoft now officially refers to these encodings as "Windows code pages".]*
- Zach initiated a discussion on compile-time vs run-time encodings. Section 4.1 describes a scenario in which file paths are pasted into source code as string literals, but the existing interpretation of such strings, when used as paths at run-time, depends on run-time locale settings.
- Peter mentioned that the Microsoft compiler now supports a `/utf-8` option that purports to define the source and execution character encodings. However, that option really only affects how literals from the source code are translated to the execution character encoding (UTF-8 at compile time, but never UTF-8 at run-time (at least, not until the newly introduced beta support in Windows 10 that requires the user to opt in)).
- Tom stated that we can't fix the compile-time vs run-time aspects of the execution character encoding.
- Martinho countered that `char8_t` offers a solution for this - we know the compile-time and run-time encoding of `char8_t` characters and strings.
- Tom suggested a response to the author: maintain consistency with existing code; `char` means "ANSI" encoding. Use `char8_t` for UTF-8 (follow the changes to path proposed in the `char8_t` proposal).
- Tom, Zach, and JeanHeyd all noted the presence of `#ifdefs` surrounding the `wchar_t` based interfaces in the proposed design. We don't use `#ifdef` as specification for implementation defined features.
- JeanHeyd noted that that `path_view` should not fight with the platform; don't propagate implementation defined behavior through interfaces to the programmer.
- Martinho observed that there is no rationale for providing `wchar_t` based interfaces only for Windows; they are perfectly applicable to other platforms as well.
- Zach stated that `path_view` should work the same as `path`; just as `string_view` does for `string`. `path_view` should support the same set of constructors that `path` has and they should behave the same. If there is a need for new constructors, they should be added to both `path` and `path_view`.
- Zack noted that `path_view` should be explicitly constructible from `path`, not the other way around. *[Editor's note: as currently specified, `path_view` is constructible from `path`, though the constructor isn't explicit. Note that `string_view`'s corresponding constructor is also not explicit.]*
- Further discussion regarding memory allocation and the behavior of the proposed `c_str` class ensued. *[Editor's note: few details of this discussion were recorded. From what I recall, consensus was that the memory allocation behavior should be implementation defined.]*
- JeanHeyd asked how we should communicate our feedback to the author.
- Zach replied with a preference for a direct person-to-person response.
- JeanHeyd volunteered to deliver feedback.
- Poll: Use execution character encoding for char interfaces, `char8_t` for UTF-8?
 - Unanimous consent.
- [P0882R0](#) - User-defined Literals for `std::filesystem::path`
 - Tom stated that SG16 concerns are limited to encoding issues; LEWG should address any other concerns; e.g., naming.
 - Peter noted that the paper punts on UTF-8 support pending a solution from the committee for differentiating ordinary and UTF-8 string literals. Fortunately, we have a solution for that in the works!
 - It was asked why the UDLs are not `constexpr`; the answer is because they produce `path` objects and the `path` constructor allocates.
 - Mark asked if the UDLs should produce `path_view` objects ala [P1030](#) above and was rewarded with a round of yeses.

- Peter observed that the UDL names are very generic (ha ha) and that the literal namespace proposed for them differs unnecessarily from existing precedent (e.g., `std::filesystem::literals` vs `std::literals::filesystem`. *[Editor's note: This design also results in the UDL declarations being visible following using namespace std::filesystem; this may be intentional.]*
 - Poll: Contingent upon adoption of ``char8_t``, add ``char8_t`` based overloads?
 - Unanimous consent.
 - [P0540R1](#) - A Proposal to Add `split/join` of `string/string_view` to the Standard Library
 - Tom observed that the paper number and filename do not match. *[Editor's note: Tom followed up with Hal and the author.]*
 - Everyone in unison, "non-member functions please!"
 - Tom asked if there were any concerns about `split/join` functions operating at the code unit level.
 - Martinho replied, no, those are useful operations for splitting/constructing grapheme clusters.
 - Zach expressed concern about increasing the surface area of string based interfaces.
 - Poll: Does adding these additional functions complicate future efforts due to increasing the set of functionality to replicate at code point or higher levels?
- [SF F N A SA]
5 1 1 0 0
- [P0645R2](#) - Text Formatting
 - Zach requested `char8_t` overloads. *[Editor's note: Peter has been planning to work on adding `char16_t` and `char32_t` support. There is an existing issue tracking support for `char16_t` at <https://github.com/fmtlib/fmt/issues/698>. That issue notes that support for `std::numprint<char16_t>` is missing; that would presumably be an issue for `char8_t` support as well.]*
 - Zach observed that formatting only works for trivial encodings in which one code unit equals one code point; otherwise, field alignments won't match up in displayed text.
 - Martinho responded that, if a font is missing a glyph for a combining character, then the combining character will likely be displayed as a separate glyph. Text layout is required to display aligned text (e.g., depends on console, cursors, etc...).
 - Tom asked how such display concerns can be addressed; `format` is not a text display tool.
 - Zach asked how field size is specified. Code units? Code points? "Characters"?
 - Peter provided a link to an existing github issue concerning field size and UTF-8: <https://github.com/fmtlib/fmt/issues/628>.
 - Tom noted that we were out of time; we'll continue discussion next time and will invite Victor to join us.
 - Tom stated our next meeting will be scheduled for three weeks from now on June 20th. The extra week is to give everyone a break following Rapperswil.

June 20th, 2018

Draft agenda:

- Rapperswil recap. Progress!
- Continue review of P0645R2 (Text Formatting), hopefully with Victor present if he can attend.
- Review the draft D1097R0 proposal:
 - <https://github.com/rmartinho/sg16/blob/master/papers/d1097r0.md>
- Discuss what we want to learn from the Swift and WebKit developers.

Attendees:

- Corentin Jabot

- JeanHeyd Meneide
- Keld Simonsen
- Mark Zeren
- Martinho Fernandes
- Peter Bindels
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- First order of business was to ensure that papers requiring updates following the Rapperswil meeting are submitted in time for the post-Rapperswil mailing. Tom confirmed that P0482R4 had been submitted and correspondence with Hal confirmed that P1025R1 (adopted at Rapperswil) will be included in the mailing. Though not discussed in Rapperswil, Martinho plans to submit a revision of P1041 for the mailing.
- [P0645R2](#) - Text Formatting
 - Victor started us off with a brief introduction of recent changes and review in Rapperswil.
 - Victor reported having read the summary of our previous meeting and discussion of P0645.
 - Discussion resumed regarding what field widths mean for multibyte encodings and combining characters.
 - Victor asked if basing field widths on grapheme clusters would be appropriate.
 - Zach provided an example of family emojis. Consider 4 person code points separated by zero width joiners. Each person code point combined with a ZWJ is a distinct grapheme cluster, but a single glyph may be used to display all four clusters. So, grapheme clusters are not the right abstraction for field width.
 - Tom claimed that `format` should be used to format code units.
 - Peter suggested assuming one column per code point.
 - Keld asked about other libraries; are there any that use abstractions above code points for field formatting?
 - Tom stated that the competition is `printf` and `iostreams`.
 - Keld asked what ICU does.
 - Zach responded that he wasn't sure, but that Python uses code points for field formatting.
 - Discussion then moved on to other topics briefly.
 - Zach expressed enthusiasm for `format_to_n`.
 - Tom asked if mixed character encodings are supported. For example:
 - `format("{} ", u"text"); // execution character encoding for format string with UTF-16 argument.`
 - Victor stated that mixed encodings are not supported and result in compilation failure.
 - Zach observed that, if `char8_t` overloads were added, that, internally, `format` must consume code points.
 - Tom responded that this is true for any multibyte encoding, and therefore true in general for the execution and wide character encodings.
 - Victor agreed, but noted that operations other than fill and field formatting could be optimized to avoid looking at code points.
 - Peter asked if any multibyte encodings allow a NUL byte in trailing code unit sequences. No such encodings were named.
 - Peter observed that, if an encoding library is used, `format` can always just read code points.
 - Zach offered to provide Victor code using code point iterators from `Boost.Text` that could be used to prototype code point based approaches.
 - Discussion briefly turned to portability of `wchar_t` and Keld's work to increase the number of C interfaces that do not rely on global program state; e.g., locale data. Keld wants to improve support for working with multiple encodings in a single process.

- Tom noted that such improvements are useful for our ideas around use of compile-time known internal encodings with transcoding to run-time determined encodings at program borders.
- Tom asked how `format` handles signed and unsigned char; are they treated as integral/arithmetic or character types?
- Victor replied that he didn't recall and would have to check.
- Keld asked about reentrancy.
- Victor responded that the only global state references are for locale data.
- Keld recommended allowing strings to be tagged with encoding data.
- Tom tried to bring discussion back to fill operations and field widths; are we agreed on use of code points for field fill/alignment?
- Martinho asked how a code point approach works when writing to a fixed width buffer (of code units).
- Victor mentioned that `format_to_n` takes a code unit count constraint.
- Peter observed that a code unit count constraint can result in truncated code unit sequences.
- Victor suggested that `format_to` could produce code points instead.
- Steve asked how to avoid writing broken code; code points produced are likely going to be written to a code unit buffer anyway.
- Keld stated that programmers like to write both code unit and code point code; perhaps both should be supported.
- Martinho claimed that truncated code unit sequences are probably not a large concern; buffers are generally larger than required anyway.
- Discussion again drifted towards encodings that are known at compile-time vs run-time.
- Keld asked what types are generally used for double byte character sets; Japanese, Chinese, ...
- Martinho responded that those tend to be variable length encodings that switch between single byte and multibyte.
- Tom agreed and mentioned ISO-2022 and escape sequences.
- Discussion drifted back to code units vs code points.
- Zach suggested that programmers will expect the output encoding to match the format string, but that code points are more consistent and natural. If the `n` in `format_to_n` means something different than for field widths, that will be a problem.
- Victor agreed that programmers will expect to be filling a code unit based buffer.
- Tom observed that more discussion would be useful, but that we need to move on.
- Zach recommended trying to support both code unit and code point based approaches and observe feedback and usage.
- [D1097R0](#) - Named character escapes
 - Martinho started by requesting feedback on:
 - name matching (currently more limited than described by [UAX44-LM2](#))
 - lack of support for named character sequences.
 - Tom recommended adding a small section that summarizes what is actually proposed. At present, the paper presents a number of options, but one must read the proposed wording to determine which options are actually proposed.
 - Tom expressed a preference for following the UAX44-LM2 rule for name matching.
 - Martinho responded with a dislike for the U+1180 HANGUL JUNGSEONG O-E exception and noted that none of the other languages he surveyed use UAX44-LM2 for matching.
 - Keld noted existing APIs that allow specifying precision for matching.
 - Martinho clarified that general collation APIs don't apply here (because of the U+1180 HANGUL JUNGSEONG O-E exception).
 - Tom asked if we should propose this for C and everyone responded yes.
 - Tom mentioned the paper should address the potential for code breakage. `"\N"` has a meaning now (it means `"N"`).
 - Tom asked if it is permissible to construct these escapes using macro concatenation.
 - Tom observed that `'_'` seemed to be missing in the definition of `c_char`.
 - Martinho stated that is intentional; `'_'` would be needed for UAX44-LM2 matching, but that actual character names never use `'_'`.

- Zach suggested adding a Tony Table to compare use of \u and \N{} escapes.
- Tom suggested clarifying that \N{} escapes would not be permitted in identifiers.
- Tom asked about interaction with raw string literals; r-char-sequence doesn't seem to include universal-character-name.
- Martinho responded that universal-character-name escapes are not recognized in raw string literals; following existing precedent.
- Rapperswil recap:
 - Tom asked if Rapperswil attendees were able to connect with authors of previously discussed papers in order to deliver our feedback.
 - JeanHeyd reported that connections did not happen. However:
 - P1030 was not discussed in Rapperswil.
 - P0882 was discussed in LEWG but not well received. No need for follow up.
 - P0540 was discussed but LEWG feedback matched ours. No need to follow up.
- We ran out of time to discuss what we want to learn from the Swift and WebKit developers.
- Tom asked about renaming the SG16 mailing list from unicode to sg16-unicode. Both Tom and Martinho had been annoyed by the similarity to the unicode.org mailing list by the same name. No objections were raised; Tom will follow up with Keld.
- Tom noted that our next regularly scheduled meeting would fall on July 4th, a US holiday. The next meeting will be scheduled for July 11th.